CYLite

Intermediate Code Generation (Three Address Code)

Course: CS327 Compilers
Assignment: Lab Assignment 4
Project: CYLite (YAPL Extension)
Repository: compilers-cylite

Team Members

Tejas Lohia (23110335)
Shardul Junagade (23110297)
Swayam Borate (23110066)

Submission Date: April 18, 2026

Contents

1 Introduction **2**

2 Implementation Flow and Topic-Wise Explanation **2**
2.1 1. IR Architecture: Quadruples and Helper APIs 2
2.2 2. Lexical Value Propagation from Tokens 3
2.3 3. Semantic Value Flow Across Grammar 3
2.4 4. Expression Evaluation and TAC Emission 4
2.5 5. Assignment Handling and LHS Safety 5
2.6 6. Control-Flow Lowering with Labels and Jumps 6
2.7 7. Output and Diagnostics Flow 7

3 Part 3 (3 marks): Code Output Format **8**
3.1 Chosen Testcase 8
3.2 Observed Generated IR (from tool output) 9
3.3 Requirement-wise Explanation 9
3.4 Result 9

4 Part 4 (3 marks): Error Handling **10**
4.1 Observed Error Behavior (invalid-input run) 10
4.2 Requirement-wise Explanation 10
4.3 Result 11
4.4 Batch Result on Regenerated Testcases 11

5 Build and Reproduction Steps **11**

6 Conclusion **11**

1 Introduction

Lab Assignment 4 extends CYLite from a parser-focused front-end to a syntax-directed translator that emits Intermediate Representation (IR) in the form of Three Address Code (TAC) quadruples. While Assignment 3 established grammar support for CYLite constructs, Assignment 4 adds semantic actions to generate executable intermediate code for expressions, assignments, conditionals, and loops.

The implementation is performed directly inside `yapl.y` using Bison semantic actions. Each relevant nonterminal now propagates either identifier names or temporary variables, and each reduction emits one or more TAC instructions into a global quadruple table.

This report documents:

- architectural changes from Assignment 3 to Assignment 4,
- design of quadruple-based IR,
- translation strategy for arithmetic and control-flow constructs,
- diagnostics and robustness behavior,
- testcase-based validation.

2 Implementation Flow and Topic-Wise Explanation

This section presents Assignment 4 changes in a logical compiler flow, from IR setup to final output, instead of a file-wise breakdown. For each topic, the complete explanation is given directly below the topic.

2.1 1. IR Architecture: Quadruples and Helper APIs

- **Implementation details:** Added `struct quadruple`, global `quad_table`, counters, and helper routines `new_temp()`, `new_label()`, and `emit()` in `yapl.y`.
- **How this helps IR generation:** Provides a fixed storage format for TAC rows and a single emission API, so every semantic action appends IR in a consistent `(op, arg1, arg2, result)` form.
- **Technical impact:** `new_temp()` guarantees unique temporaries (`t1, t2, ...`) for intermediate results, and `new_label()` guarantees unique branch targets (`L1, L2, ...`) for control-flow lowering.
- **Why this is necessary:** Without a centralized quadruple table, parser actions would only validate syntax and would not preserve an ordered IR trace.

```c
struct quadruple {
    char op[10];
    char arg1[50];
    char arg2[50];
    char result[50];
};

struct quadruple quad_table[1000];
int quad_idx = 0;
int temp_count = 0;

/* Generates t1, t2, t3... */
char* new_temp() {
    char* t = (char*)malloc(10);
    sprintf(t, "t%d", ++temp_count);
    return t;
}

char* new_label() {
    char* l = (char*)malloc(10);
    sprintf(l, "L%d", ++label_count);
    return l;
}

/* Records a new row in our table */
void emit(char* op, char* arg1, char* arg2, char* res) {
    strcpy(quad_table[quad_idx].op, op);
    strcpy(quad_table[quad_idx].arg1, arg1);
    strcpy(quad_table[quad_idx].arg2, arg2);
    strcpy(quad_table[quad_idx].result, res);
    quad_idx++;
}
```

Figure 1: Quadruple representation and helper routines in `yapl.y`: global IR table, temporary generator, label generator, and `emit()` API.

2.2 2. Lexical Value Propagation from Tokens

- **Implementation details:** Updated identifier and integer-constant rules in `yapl.l` to copy `yytext` into `yylval.name` before returning tokens.
- **How this helps IR generation:** Preserves exact variable names and literal values so emitted TAC contains source-accurate operands.
- **Technical impact:** Parser actions can directly consume identifier and constant text while building quadruples.
- **Why this is necessary:** If token lexemes are not forwarded to parser semantic values, TAC cannot correctly represent program variables and constants.

```c
{L}({A})* {
    strcpy(yylval.name, yytext);
    return check_type();
}
```

```c
{HP}{H}+{IS}? |
{NZ}{D}*{IS}? |
"0"{O}*{IS}? {
    strcpy(yylval.name, yytext); /* Copy "5" or "10" to the parser */
    return I_CONSTANT;
}
```

Figure 2: Lexer updates in `yapl.l`: identifier and integer lexemes copied into `yylval.name` for TAC operands.

2.3 3. Semantic Value Flow Across Grammar

- **Implementation details:** Added `%union` with `name[50]` and assigned `%type <name>` to expression-related nonterminals in `yapl.y`.

- **How this helps IR generation:** Allows each grammar reduction to return either a symbol/literal or a temporary name, enabling parent rules to use child results directly while emitting TAC.
- **Technical impact:** Value propagation works across the full expression chain (primary → unary → multiplicative → additive → relational), which is required to emit correct operand references.
- **Why this is necessary:** If nonterminals do not carry semantic names, parent rules lose child computation results and cannot form valid TAC operands.

```
%union {
    char name[50];  /* Holds variable names or temp names like 't1' */
    int val;
    struct symtab *symp; /* Added from your second union */
}

%type <name> expression assignment_expression additive_expression multiplicative_expression
%type <name> unary_expression primary_expression cast_expression constant
%type <name> postfix_expression string generic_selection
%type <name> shift_expression relational_expression equality_expression
%type <name> and_expression exclusive_or_expression inclusive_or_expression
%type <name> logical_and_expression logical_or_expression conditional_expression
%type <name> unary_operator assignment_operator expression_statement type_name
%type <name> if_prefix while_prefix while_cond
```

Figure 3: Semantic value plumbing in `yapl.y` using `%union` and `%type <name>` for expression-level TAC propagation.

2.4 4. Expression Evaluation and TAC Emission

- **Implementation details:** Attached semantic actions to unary, arithmetic, relational, equality, and logical expression rules; each action creates temporaries and calls `emit()`.
- **How this helps IR generation:** Converts parse-tree computations into explicit TAC steps, where intermediate results are materialized as temporaries and reused in higher-level expressions.
- **Technical impact:** Operator precedence is preserved naturally by grammar structure, and each reduction emits exactly one computation step, making the final quadruple sequence deterministic and debuggable.
- **Example effect:** For `c = a + b * 5`, multiplication is emitted first into `t1`, then addition into `t2`, then assignment `c = t2`.

```
unary_expression
    : postfix_expression { strcpy($$, $1); }
    | INC_OP unary_expression
    {
        char* t = new_temp();
        emit("+", $2, "1", t);   /* Increment the value */
        emit("=", t, "", $2);    /* Update the original variable */
        strcpy($$, $2);          /* Return the updated variable name */
    }
    | DEC_OP unary_expression
    {
        char* t = new_temp();
        emit("-", $2, "1", t);   /* Decrement the value */
        emit("=", t, "", $2);    /* Update the original variable */
        strcpy($$, $2);
    }
    | unary_operator cast_expression
    {
        char* t = new_temp();
        /* The PDF example explicitly uses "minus" for unary negation  */
        if (strcmp($1, "-") == 0) {
            emit("minus", $2, "", t);
        } else {
            emit($1, $2, "", t);
        }
        strcpy($$, t);
    }
    | SIZEOF unary_expression
    {
        char* t = new_temp();
        emit("sizeof", $2, "", t);
        strcpy($$, t);
    }
    | SIZEOF '(' type_name ')'
    {
```

Figure 4: Expression lowering rules in `yapl.y`: arithmetic/relational/logical operations emitting TAC with temporaries.

2.5 5. Assignment Handling and LHS Safety

- **Implementation details:** Enhanced assignment actions to validate the target and force the final `=` emission to the original LHS identifier.
- **How this helps IR generation:** Preserves assignment semantics (e.g., `a = E`) by preventing incorrect IR forms where a temporary is mistakenly treated as the final destination.
- **Technical impact:** Handles simple assignment and compound assignment uniformly, and protects against erroneous LHS lowering caused by unary/rewrite side effects.
- **Why this is necessary:** The final store must always target program variables (e.g., `a`), otherwise generated TAC can become semantically invalid even if parsing succeeds.

```
assignment_expression
    : conditional_expression
    {
        strcpy($$, $1);
    }
        | unary_expression assignment_operator assignment_expression
        {
                char target[50];
                strcpy(target, $1);

                // VALIDATION CHECK: Eliminate unary ops on LHS
                if (target[0] == 't' || target[0] == 'T') {
                    for (int i = quad_idx - 1; i >= 0; i--) {
                        if (strcmp(quad_table[i].result, target) == 0) {
                            if (strcmp(quad_table[i].op, "minus") == 0 || strcmp(quad_table[i].op, "-") =
                                strcpy(target, quad_table[i].arg1);
                                for (int k = i; k < quad_idx - 1; k++) {
                                    quad_table[k] = quad_table[k + 1];
                                }
                                quad_idx--;
                                temp_count--; // Decrement temp to maintain sequential count
                            }
                            break;
                        }
                    }
                }

                if (strcmp($2, "=") == 0) {
                        emit("=", $3, "", target);
                } else {
                        char op[3];
                        strncpy(op, $2, strlen($2)-1);
                        op[strlen($2)-1] = '\0';
                        char* t = new_temp();
                        emit(op, target, $3, t);
                        emit("=", t, "", target);
```

Figure 5: Assignment translation logic with LHS safety validation to preserve correct final destination in emitted TAC.

2.6 6. Control-Flow Lowering with Labels and Jumps

- **Implementation details:** Added helper nonterminals and actions for conditionals/loops that emit `ifFalse`, `goto`, and `LABEL` instructions.
- **How this helps IR generation:** Lowers structured control flow into branch-and-label TAC, producing an explicit control-flow graph that can be analyzed or optimized later.
- **Technical impact:** The grammar now emits entry, body, increment, and exit labels for loops and emits true/false branch structure for conditionals, including if-elif-else chains. In the latest update, `goto` and `LABEL` targets are emitted in the `result` field, keeping control-flow quadruples schema-consistent with readable output.
- **Why this is necessary:** High-level constructs cannot be consumed by later compiler stages directly; they must be transformed into explicit jump-based flow in IR.

```
if_prefix
    : IF '(' expression ')'
    {
        char *l_false = new_label();
        emit("ifFalse", $3, "", l_false);
        strcpy($$, l_false); /* FIX: strcpy instead of = */
    }
    ;

while_prefix
    : WHILE
    {
        char *l_start = new_label();
        emit("LABEL", l_start, "", "");
        strcpy($$, l_start); /* FIX: strcpy */
    }
    ;

while_cond
    : while_prefix '(' expression ')'
    {
        char *l_end = new_label();
        emit("ifFalse", $3, "", l_end);
        sprintf($$, "%s %s", $1, l_end); /* Pack start and end labels */
    }
    ;

selection_statement
    /* RULE 1: IF-ELSE */
    : if_prefix statement ELSE
      {
          char *l_end = new_label();
          emit("goto", l_end, "", "");
          emit("LABEL", $1, "", ""); /* $1 is the label from if_prefix */

          ladder_len++;
          $<val>$ = (ladder_len - 1);
```

Figure 6: Control-flow TAC generation for conditionals and loops using `ifFalse`, `goto`, and `LABEL`.

2.7 7. Output and Diagnostics Flow

- **Implementation details:** Added `print_quad_table()` and updated `main()` to print source, diagnostics, and generated TAC. The current implementation uses `format_quad_statement()` to derive a readable `statement` column from `(op, arg1, arg2, result)`.
- **How this helps IR generation:** Makes IR output directly visible for validation, helping confirm that semantic actions produce the expected quadruples per testcase.
- **Technical impact:** The output order now clearly mirrors the compilation pipeline: input program → parser success and counters → TAC table → reverse derivation. The generated table remains aligned for both expression and control-flow rows.
- **Why this is necessary:** Assignment evaluation requires demonstrable generated IR, and this output format provides direct evidence for correctness.

```c
void print_quad_table() {
    printf("\n--- Generated Intermediate Code (Quadruples) ---\n");
    printf("%-10s | %-10s | %-10s | %-10s\n", "op", "arg1", "arg2", "result");
    printf("------------------------------------------------------------\n");
    for(int i = 0; i < quad_idx; i++) {
        printf("%-10s | %-10s | %-10s | %-10s\n",
               quad_table[i].op, quad_table[i].arg1,
               quad_table[i].arg2, quad_table[i].result);
    }
}

int main(int argc, char **argv)
{
    extern FILE *yyin;
    int i;
    const char *input_file = NULL;
    const char *svg_file = NULL;

    if (argc == 2 && (strcmp(argv[1], "--help") == 0 || strcmp(argv[1], "-h") == 0))
    {
        printf("Usage: ./yapl <input_file> [--svg out.svg]\n");
        printf("\n");
        printf("Options:\n");
        printf("  -h, --help       Show this help message and exit\n");
        printf("  --svg <path>     Write reverse derivation graph as SVG\n");
        return 0;
    }

    for (i = 1; i < argc; i++)
    {
        if (strcmp(argv[i], "--svg") == 0)
        {
            if (i + 1 >= argc)
            {
                sprintf(buff, "***process terminated*** [input error]: missing path after --svg");
                mode = 1;
                yyerror(buff);
```

Figure 7: Driver-side reporting in `main()`: source echo, parser diagnostics, and tabular quadruple output.

Error reporting remains robust through verbose parser diagnostics and guarded input handling, ensuring malformed inputs terminate cleanly with meaningful messages rather than causing unstable behavior.

3 Part 3 (3 marks): Code Output Format

This section demonstrates output-format compliance using testcase `testcases_a4/tc1.c`.

3.1 Chosen Testcase

```c
int main() {
    int a; int b; int c;
    a = 10;
    b = 20;
    c = a + b * 5;
}
```

3.2 Observed Generated IR (from tool output)

statement	op	arg1	arg2	result
a=10	=	10		a
b=20	=	20		b
t1=b*5	*	b	5	t1
t2=a+t1	+	a	t1	t2
c=t2	=	t2		c

3.3 Requirement-wise Explanation

1. **Intermediate code printed row-by-row from top to bottom:** The IR appears in execution order. First constants are assigned to `a` and `b`, then multiplication is emitted, then addition, and finally assignment to `c`. This matches top-down TAC generation.
2. **Readable temporary variable naming (`t1`, `t2`, ...):** Intermediate values are stored in `t1` and `t2`, making data flow easy to follow.
3. **Clear source code and IR separation:** Output prints source under the input-program header and prints TAC under the generated-intermediate-code header, so both phases are clearly separated.

3.4 Result

The implementation satisfies all three Part 3 requirements. The generated output is tabular, ordered, and readable, which supports both debugging and evaluation.
After normalizing control-flow emission, the generated IR now shows branch targets in a consistent way. For example, in control-flow test outputs, rows appear as `goto L2` and `LABEL L1` with the label stored in the `result` column, which keeps output formatting and interpretation unambiguous.

```
--- Input Source Program ---
int main() {
    int a; int b; int c;
    a = 10;
    b = 20;
    c = a + b * 5;
}

---------------------------

*** Parsing Successful ***
#global_declarations = 1
#function_definitions = 1
#integer_constants = 3
#pointers_declarations = 0
#ifs_without_else = 0
if-else max-depth = 0

--- Generated Intermediate Code (Quadruples) ---
statement          | op         | arg1       | arg2       | result
-------------------------------------------------------------------------------
a=10               | =          | 10         |            | a
b=20               | =          | 20         |            | b
t1=b*5             | *          | b          | 5          | t1
t2=a+t1            | +          | a          | t1         | t2
c=t2               | =          | t2         |            | c
```

Figure 8: row-wise TAC table with readable temporaries and separated source/IR blocks.

4 Part 4 (3 marks): Error Handling

This section explains diagnostic behavior during code generation and parsing.

4.1 Observed Error Behavior (invalid-input run)

To verify robustness on incorrect input, the parser was run on:

```
int main() {
    int a;
    a = b + ;
    return 0;
}
```

Observed diagnostic:

```
***parsing terminated*** [syntax error]
line 3, column 13: unexpected token ";"
```

4.2 Requirement-wise Explanation

- **Detect invalid expressions or unsupported constructs:** The malformed assignment `a = b + ;` is detected immediately and parser termination is triggered correctly.
- **Display meaningful error messages:** The diagnostic includes both line and column information and reports the unexpected token, which is useful for debugging.

- **Ensure program does not crash on incorrect input:** The parser exits gracefully with a controlled error message instead of crashing or producing corrupted IR.

4.3 Result

The implementation satisfies Part 4 expectations for practical diagnostics and runtime robustness. Invalid input is handled predictably and reported with useful location information.

4.4 Batch Result on Regenerated Testcases

Following the latest parser updates, all testcase files `tc1.c` through `tc10.c` were rerun and outputs were regenerated in `testcases_a4/tc1.txt` to `testcases_a4/tc10.txt`. All runs completed successfully and produced parsing-success diagnostics along with IR tables.

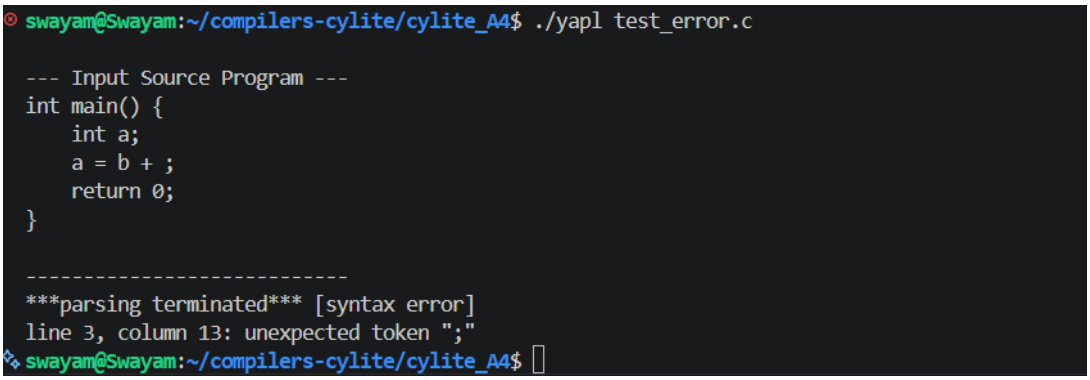

Figure 9: meaningful syntax-error reporting and graceful termination on incorrect input.

5 Additional Worked Example: `tc3.c`

To further explain intermediate-code generation, this section analyzes testcase `testcases_a4/tc3.c`, which contains a simple `if` condition without `else`.

5.1 Input Program

```
int main() {
    int a; int b;
    a = 5;
    b = 10;
    if (a < b) {
        a = a + 1;
    }
}
```

5.2 Observed Generated IR

statement	op	arg1	arg2	result
a=5	=	5		a
b=10	=	10		b
t1=a<b	<	a	b	t1
ifFalse t1 goto L1	ifFalse	t1		L1
t2=a+1	+	a	1	t2
a=t2	=	t2		a
LABEL L1	LABEL			L1

5.3 Step-by-Step Explanation

1. **Initialization:** The first two rows assign constants to `a` and `b`.
2. **Condition evaluation:** `t1 = a < b` computes the relational condition and stores the Boolean-style result in a temporary.
3. **Conditional branch:** `ifFalse t1 goto L1` skips the `if` body when the condition is false.
4. **If-body execution:** When the condition is true, `t2 = a + 1` and then `a = t2` implement the update.
5. **Join point:** `LABEL L1` marks the control-flow merge point after the `if` block.

5.4 Result

This testcase confirms correct translation of a single-branch `if` statement into TAC: condition computation, conditional jump, body instructions, and final label placement are all emitted in proper execution order.

6 Build and Reproduction Steps

```
cd cylite_A4
make clean
make
./yapl testcases_a4/tc1.c
```

7 Conclusion

Assignment 4 successfully transforms CYLite into a working intermediate code generator using TAC quadruples. Compared to Assignment 3, the parser now carries semantic values and emits executable IR for expressions and control flow, while preserving detailed diagnostics and stable behavior on invalid input.
The implementation demonstrates end-to-end translation from grammar reductions to structured quadruple output with labels, temporaries, and branch instructions. This establishes a clean base for subsequent compiler phases such as optimization and target code generation.